

DIPLOMATURA EN MACHINE LEARNING CON PYTHON

Redes Neuronales

Redes Neuronales

- Instalación y uso del paquete
- Ejemplo conceptual en Excel
- Ejemplo en Python
- Principales parámetros de ajuste y control
- Problema concreto

Un poco de contenido teórico:

¿Qué es una red neuronal? El cerebro humano consiste en 100 billones de células llamadas neuronas, conectadas entre ellas mediante sinapsis. Si hay suficientes sinapsis de entrada, la neurona también funcionara. A este proceso lo denominamos pensar. Este proceso se puede modelar creando una red neuronal en la computadora. No es necesario modelar la complejidad biológica de la neurona, solo modelar las reglas a un alto nivel.

Se utilizan matrices, que son representados por una grilla de números. Para hacerlo simple, se modelara una neurona simple con tres entradas y una salida.

Se va a entrenar a la neurona para resolver el problema que se presenta líneas abajo. Los primeros cuatro ejemplos son denominados el **set de entrenamiento**. El resultado, de acuerdo al problema, debería ser "0" o "1".

Ejemplo conceptual a probar en Python

	Entrada			Salida
Ejemplo 1	0	0	1	0
Ejemplo 2	1	1	1	1
Ejemplo 3	1	0	1	1
Ejemplo 4	0	1	1	0
Nuevo Escenario	1	0	0	¿?

Si nos fijamos el resultado esperado es 1. ¿Por qué?

	Entrada			Salida
Ejemplo 1	0	0	1	0
Ejemplo 2	1	1	1	1
Ejemplo 3	1	0	1	1
Ejemplo 4	0	1	1	0
Nuevo Escenario	1	0	0	¿?

En la salida se espera el valor de la primera columna.

Proceso de Entrenamiento

¿Cómo enseñaremos a nuestra neurona a responder correctamente?

Le daremos a cada entrada un peso, que puede ser un número positivo o negativo. Una entrada con un gran peso positivo o un gran peso negativo, tendrá un fuerte efecto en la salida de la neurona. Antes de empezar, estableceremos cada peso como un número aleatorio. Luego empezaremos el proceso de entrenamiento:

- Tomar las entradas de los valores de entrenamientos, ajustar por los pesos y pasarlas a través de una fórmula para calcular la salida de las neuronas.

- Calcular el error, que es la diferencia entre la salida de la neurona y la salida deseada en el ejemplo del conjunto de entrenamiento.
- Dependiendo de la dirección del error, ajuste los pesos ligeramente
- Repetir este proceso 10 mil veces.

Los pesos de la neurona alcanzarán un óptimo para el conjunto de entrenamiento. Luego, si permitimos que la neurona piense en una nueva situación, que siga el mismo patrón, debería hacer una buena predicción. Este proceso se llama propagación.

Fórmula para calcular la salida de la neurona

Primero tomamos la suma ponderada de las entradas de la neurona, que es:

$$\sum \text{Peso } i \dots \text{Salida } i = \text{Peso } 1 * \text{Salida } 1 + \text{Peso } 2 * \text{Salida } 2 + \dots$$

A continuación normalizamos la entrada, considerando que tenemos un resultado que se encuentra entre 0 y 1. Para esto seleccionamos una función matemática conveniente, denominada función Sigmoid:

$$\frac{1}{1 + e^{-x}}$$

Si se sustituye la primera ecuación en la segunda, la fórmula final de la neurona sería:

$$\text{Salida de la Neurona} = \frac{1}{1 + e^{-\sum \text{Peso } i \dots \text{Salida } i}}$$

Fórmula para ajustar los pesos

Durante el ciclo de entrenamiento es cuando se ajustan los pesos.

¿Cómo se ajustan los pesos?

Podemos utilizar la fórmula «Derivada ponderada del error».

Ajustar pesos = error * entrada * Gradiente de la Curva Sigmoid (salida)

¿Por qué esta fórmula?

Se requiere que el ajuste sea proporcional al tamaño del error. Entonces se multiplica por la entrada que es un 0 o un 1. Si la entrada es 0, el peso no se ajusta. Luego, multiplicamos por el gradiente de la curva de la función Sigmoid. Para comprender esto último, considere:

1. Utilizamos la curva de la función Sigmoid para calcular la salida de la neurona.
2. Si la salida es un número grande, positivo o negativo, esto significa que la neurona está bastante segura para un lado o para el otro.
3. La función Sigmoid tiene un gradiente que tiende a cero para números grandes
4. Si la neurona confía en que el peso existente es correcto, no va a querer sobre ajustarlo. Multiplicando por el gradiente de la curva Sigmoid se logra esto.

5. El gradiente de la curva Sigmoid se puede obtener de la **derivada** de la función:

Gradiente de la Curva Sigmoid (salida) = salida * (1-salida)

Reemplazando la primera ecuación en la segunda, la formula final para el ajuste de los pesos es:

Ajustar pesos = error * entrada* salida * (1- salida)

Hay otras formulas alternativas, que permiten a la neurona aprender más rápido, pero la ventaja de esta fórmula es que es sencilla de implementar.

Construyendo el código Python – Instalación y uso de paquetes

Aunque no utilizaremos una biblioteca de redes neuronales, importaremos cuatro métodos de la librería NumPy de Python, estas son:

1. **exp:** la función exponencial
2. **array:** permite crear matrices
3. **dot:** multiplica matrices
4. **random:** proporciona números aleatorios

```
from numpy import exp, array, random, dot

class NeuralNetwork():
    def __init__(self):
        # Seed el generador de números aleatorios, con una raíz a fin de generar
        # los mismo números aleatorios en cada proceso.
        random.seed(1)
        # Modelamos una sola neurona, con 3 conexiones de entrada y una conexión de
        salida
        # Asignamos los pesos aleatorios a una matriz de 3x1, con valores en el rango de -1
        a 1
        # y significa 0
        self.synaptic_weights = 2 * random.random((3, 1)) - 1

        # La función Sigmoid, que describe una curva en forma de S
        # Se pasan la suma ponderada de las entradas a través de esta función para
        # normalizarlos entre 0 y 1.
        def __sigmoid(self, x):
            return 1 / (1 + exp(-x))

        # La derivada de la función Sigmoid.
        # Este es el gradiente de la función Sigmoid.
        # Indica la confianza que tenemos en el peso existente.
        def __sigmoid_derivative(self, x):
            return x * (1 - x)
```

```
# Entrenamos a la red neuronal a través de un proceso de prueba y error
# Se realiza un ajuste de los pesos sinápticos cada vez.
def train(self, training_set_inputs, training_set_outputs,
number_of_training_iterations):
    for iteration in range(number_of_training_iterations):
        # Pasar el conjunto de entrenamiento a través de nuestra red neuronal (una sola neurona)
        output = self.think(training_set_inputs)

        # Calcular el error(La diferencia entre el resultado deseado y el resultado obtenido).
        error = training_set_outputs - output

        # Multiplica el error por la entrada y nuevamente por el gradiente de la curva Sigmoid
        # Esto significa que los pesos menos confiables se están ajustando más
        # Esto significa que las entradas que son cero, no causan cambios en los pesos.
        adjustment = dot(training_set_inputs.T, error *
self.__sigmoid_derivative(output))

        # Ajuste de los pesos
        self.synaptic_weights += adjustment

# La red neuronal piensa.
def think(self, inputs):
    # Pasar las entradas a través de nuestra red neuronal (una neurona)
    return self.__sigmoid(dot(inputs, self.synaptic_weights))

if __name__ == "__main__":

    #Iniciar una red neuronal de una neurona
    neural_network = NeuralNetwork()

    print ("Random starting synaptic weights: ")
    print (neural_network.synaptic_weights)

    # El conjunto de pruebas. Tenemos cuatro ejemplos, cada uno consiste en 3 valores de entrada y un valor de salida.
    training_set_inputs = array([[0, 0, 1], [1, 1, 1], [1, 0, 1], [0, 1, 1]])
    training_set_outputs = array([[0, 1, 1, 0]]).T

    # Entrenar la red neuronal utilizando el conjunto de entrenamiento.
    # Realizar 10000 veces y realizar un ajuste más pequeño cada vez.
    neural_network.train(training_set_inputs, training_set_outputs, 10000)

    print ("New synaptic weights after training: ")
    print (neural_network.synaptic_weights)
```

```
# Prueba la red neuronal con una nueva situación
print ("Considering new situation [1, 0, 0] -> ? : ")
print (neural_network.think(array([1, 0, 0])))
```

Salida:

```
C:\Users\susan\OneDrive\Documentos\Python>python RedNeuronal.Py
Random starting synaptic weights:
[[-0.16595599]
 [ 0.44064899]
 [-0.99977125]]
New synaptic weights after training:
[[ 9.67299303]
 [-0.2078435 ]
 [-4.62963669]]
Considering new situation [1, 0, 0] -> ? :
[0.99993704]
```

Obtengo casi 1.-

Si pruebo con 1000000 reemplazando en el parámetro

```
neural_network.train(training_set_inputs, training_set_outputs, 1000000)

print ("New synaptic weights after training: ")
print (neural_network.synaptic_weights)

# Prueba la red neuronal con una nueva situación
print ("Considering new situation [1, 0, 0] -> ? : ")
print (neural_network.think(array([1, 0, 0])))
```

Obtengo lo siguiente:

```
C:\Users\susan\OneDrive\Documentos\Python>python redneuronal.Py
Random starting synaptic weights:
[[-0.16595599]
 [ 0.44064899]
 [-0.99977125]]
New synaptic weights after training:
[[16.62742752]
 [-0.20290209]
 [-8.11084696]]
Considering new situation [1, 0, 0] -> ? :
[0.99999994]
```

Más aproximado a 1.